CS-3510-B Problem Set 7

Elijah Martin McCauley

TOTAL POINTS

100 / 100

QUESTION 1

1 Problem 1 20 / 20

✓ **- 0 pts** *Correct*

- 20 pts Completely Incorrect/ Missing

- 5 pts Incorrect/ Missing Algorithm for a)

- 2 pts Incorrect/ Missing runtime analysis for a)

- 5 pts Incorrect/ Missing Algorithm for b)

- 2 pts Incorrect/ Missing runtime analysis for b)

- 5 pts Incorrect/ Missing Algorithm for c)

- 2 pts Incorrect/ Missing runtime analysis for c)

- 5 pts Incorrect/ Missing Algorithm for d)

- 2 pts Incorrect/ Missing runtime analysis for d)

QUESTION 2

2 Problem 2 20 / 20

✓ **- 0 pts** *Correct*

- 20 pts Completely Incorrect (Proved NP-C)/ Non poly-time algorithm provided/ Missing

- 5 pts Minor Error

- 15 pts Major Error (ex. Incorrect reduction to 2-SAT/ More than 2 edge cases exist)

QUESTION 3

3 Problem 3 30 / 30

✓ **- 0 pts** *Correct*

- 30 pts Completely Incorrect/ Missing

Proof of NP

- 5 pts Incorrect/ Missing NP proof

- 3 pts Major Error/Incomplete

- 0 pts No runtime specified

- 2 pts Minor error

- 20 pts Reverse reduction provided (2/3 SAT $$\rightarrow$$ NP-complete problem)

Reduction

- 15 pts Incorrect/Missing Reduction

- 5 pts Minor Mistake (ex. small mistake in converting SAT to 2/3 SAT, Direction of the reduction not provided)

- 10 pts Major Error (ex. reduction cannot be justified)

Bi-directional Proof

- 5 pts Incorrect/ Missing $$x \in A \implies f(x) \in B$$

- 2 pts Minor Error (i.e. Incomplete explanation)

- 4 pts Major Error (i.e. forgot to prove it for all cases)

- 5 pts Incorrect/ Missing $$x \notin A \implies f(x) \notin B$$

- 2 pts Minor Error (i.e. Incomplete explanation)

- 4 pts Major Error (i.e. forgot to prove it for all cases)

QUESTION 4

4 Problem 4 30 / 30

✓ **- 0 pts** *Correct*

- **30 pts** Completely Incorrect/ Missing

Proof of NP

- **5 pts** Incorrect/ Missing NP proof
- **3 pts** Major Error
- **2 pts** Minor Error
- **20 pts** Reverse reduction provided (Modified-4-SAT $$\rightarrow$$ NP-complete problem)

Reduction

- **15 pts** Incorrect Reduction
- **5 pts** Minor Mistake (ex. small mistake in converting 3-SAT to Modified-4-SAT)
- **10 pts** Major Error (ex. reduction cannot be justified)

Bi-directional Proof

- **5 pts** Incorrect/ Missing $$x \in A \implies f(x) \in B$$
- **2 pts** Minor Error (i.e. Incomplete explanation)
- **4 pts** Major Error (i.e. forgot to prove it for all cases)
- **5 pts** Incorrect/ Missing $$x \notin A \implies f(x) \notin B$$
- **2 pts** Minor Error (i.e. Incomplete explanation)
- **4 pts** Major Error (i.e. forgot to prove it for all cases)

gradescope

Problem 1

(20 points)

Let's define a `Max-Weight Tree` problem that takes as input a weighted, connected, and undirected graph $G = (V, E)$ and outputs a maximum spanning tree of G. Show that `Max-Weight Tree` problem is in class NP. Consider a solution S provided to you as a subset of edges.

Here are the following steps to show if `Max-Weight Tree` problem is in NP. For each of them, provide a polynomial-time algorithm.

a.) Verify that the subgraph induced by S is connected.

b.) Verify that the subgraph induced by S is cycle-free.

c.) Verify that the subgraph induced by S has vertex set V.

d.) Finally, verify that the subgraph induced by S is a maximum spanning tree.

Make sure to write the time complexity for each step.

Solution:

a) We can verify that a subgraph induced by S is connected by running DFS starting at a random node on the subgraph. If we reach all the nodes in the subgraph, then it is connected. This will run in $\mathcal{O}(|S| + |V|)$ time where S is the edges of the subgraph and V is the verticies.

b) We can verify that the subgraph induced by S is cycle-free by running either BFS or DFS on the subgraph starting at a random node. If we keep track of the visited nodes, then if we encounter a node which has two visited nodes connected to it (including its parent node), that means that there is a cycle because there are multiple ways to get to that node: from its parent and from the other visited node. Because this is running DFS or BFS, it will run in $\mathcal{O}(|S| + |V|)$ time where S is the edges of the subgraph and V is the verticies.

c) To verify that the subgraph induced by S has a vertex set V, we can loop over the entries of S and record a list of unique verticies. We can then simply compare the length of this list and the length of vertex set V, if the list has too few entries, then it does not have vertex set V, but if it has the same number then it does. This will run in $\mathcal{O}(|S|)$ because we loop over each edge which is $\mathcal{O}(|S|)$ and then just compare two integers which is $\mathcal{O}(1)$.

d) To verify that the subgraph induced by S is a maximum spanning tree, we need to run a modified Kruskal's algorithm on the original graph which chooses the maximum values instead of the minimum. We can then compare the weight of that maximum tree to the sum of the weights of S. If they are the same, then S is a maximum spanning tree. This will run in $\mathcal{O}(|E|log(|V|))$ because the most time consuming part of the algorithm is simply running the Kruskals algorithm.

1 Problem 1 20 / 20

✓ **- 0 pts** *Correct*

- 20 pts Completely Incorrect/ Missing

- 5 pts Incorrect/ Missing Algorithm for a)

- 2 pts Incorrect/ Missing runtime analysis for a)

- 5 pts Incorrect/ Missing Algorithm for b)

- 2 pts Incorrect/ Missing runtime analysis for b)

- 5 pts Incorrect/ Missing Algorithm for c)

- 2 pts Incorrect/ Missing runtime analysis for c)

- 5 pts Incorrect/ Missing Algorithm for d)

- 2 pts Incorrect/ Missing runtime analysis for d)

gradescope

Problem 2
(20 points)

Show that the following `All-trueORfalse` problem is in class P.

Input: A boolean formula in CNF with m clauses and n variables such that each clause C_i has exactly three literals.
Output: A satisfying assignment such that each of the three literals in every clause C_i, for $1 \leq i \leq m$, are either all True or all False.

Solution: To show that a problem is in class P, we need to show that it can be reduced to another problem in class P in polytime, for this problem we will use 2sat. For each clause in All-trueORfalse, of the form $(X_1 \vee X_2 \vee X_3)$, we can rewrite as $((X_1 \vee \neg X_2) \wedge (X_2 \vee \neg X_3) \wedge (X_3 \vee \neg X_1))$. This is because the truth value for each of the new 3 clauses will be true if and only if X_1, X_2, and X_3 are all true or all false which would make the 2sat run on these new clauses true; otherwise one of the three new clauses will be false which would make 2SAT false. This new format has only clauses with 2 literals, so we can use 2SAT which is in P, therefore All-trueORfalse is also in P. This change of form only takes looping over each clause and rewriting it, so it would be $\mathcal{O}(m)$ which is polytime.

2 Problem 2 **20 / 20**

✓ **- 0 pts** *Correct*

- 20 pts Completely Incorrect (Proved NP-C)/

Non poly-time algorithm provided/ Missing

- 5 pts Minor Error

- 15 pts Major Error (ex. Incorrect reduction to 2-SAT/ More than 2 edge cases exist)

gradescope

Problem 3
(30 points)

Show that the following `TwoThirdsMajority-SAT` problem is NP-Complete.

Input: a boolean formula in CNF with n variables and m clauses, where m is divisible by 3.
Output: A satisfying assignment, if exists, such that at least 2/3 of the clauses in the given input CNF evaluates to True.

Note: Write the reduction in the format described on the front page of this document.

Solution: To prove that TwoThirdsMajority-SAT is NP-Complete, we need to prove it is both in NP and NP-Hard. Let's start with NP. The witness is a set of boolean assignments for the n variables. To prove that it is in NP, we simply need to loop over the m clauses. For each clause, we plug the corresponding assignments of the witness into the literals and determine if the clause is true or false given that assignment. As we go, we can keep a counter of clauses which are true. After looping over all m clauses and counting the number which are true, we can simply divide that number by m. If it is greater than 2/3 then the assignment satisfies, else it does not. The runtime would be $\mathcal{O}(mn)$ because we are looping over m clauses and for each clause we need to plug in at most 2n assignemnts in order to classify the clause as true or false. Keeping the counter of trues and the division at the end would take negligible time.
Now we need to prove it is NP-Hard by reducing a known NP-Complete problem to TwoThirdsMajorit-SAT in polytime. 1.) First we need to pick A, for this problem lets use SAT.
2.) To reduce SAT to TwoThirdsMajorit-SAT in polytime, we need to take the number of clauses input into SAT (which we will call c). To that set of clauses, we add c new clauses containing one literal which we will call z which is always set to true so these clauses will be true, and c new clauses only containing ¬z so each of these clauses will be false. This would run in polytime because we are simply looping over the c clauses and adding 2 clauses for each one. This means the runtime would be $\mathcal{O}(c)$ where c is the length of the input for SAT.
3.) We now must prove that $x \in A \implies f(x) \in B$. For x to satisfy A which in our case is SAT, all of the clauses must evaluate to true. After applying our reduction where we add one true and one false clause for each clause in the input to SAT, we now would have the original set of clauses which are all true, a set of clauses of equal length which are all true, and a set of clauses of equal length which are all false. Therefore, 2/3 of the clauses would be true if and only if all the clauses in the original input are true, so if x satisfies SAT, f(x) will satisfy 2/3Majority-SAT
4.) Lastly, we must prove Prove that $x \notin A \implies f(x) \notin B$. For x to not satisfy A which is SAT, it means at least one of the clauses are false. When we apply our reduction, we will not fulfill the 2/3 requirement to be true because we will have 1/3 all true, 1/3 all false, and 1/3 which is the original input which is not all true. Therefore, if x does not satisfy A, after the reduction it does not reach the 2/3 and does not satisfy B.
5.) Since SAT was NP-complete, and $SAT \leq_p 2/3Majoritysat$, we can conclude that 2/3majoritysat is NP-hard.

3 Problem 3 30 / 30

✓ **- 0 pts** *Correct*

- 30 pts Completely Incorrect/ Missing

Proof of NP

- 5 pts Incorrect/ Missing NP proof

- 3 pts Major Error/Incomplete

- 0 pts No runtime specified

- 2 pts Minor error

- 20 pts Reverse reduction provided (2/3 SAT $$\rightarrow$$ NP-complete problem)

Reduction

- 15 pts Incorrect/Missing Reduction

- 5 pts Minor Mistake (ex. small mistake in converting SAT to 2/3 SAT, Direction of the reduction not provided)

- 10 pts Major Error (ex. reduction cannot be justified)

Bi-directional Proof

- 5 pts Incorrect/ Missing $$x \in A \implies f(x) \in B$$

- 2 pts Minor Error (i.e. Incomplete explanation)

- 4 pts Major Error (i.e. forgot to prove it for all cases)

- 5 pts Incorrect/ Missing $$x \notin A \implies f(x) \notin B$$

- 2 pts Minor Error (i.e. Incomplete explanation)

- 4 pts Major Error (i.e. forgot to prove it for all cases)

gradescope

Problem 4
(30 points)

Show that the following `Modified-4-SAT` problem is NP-Complete.

Input: a boolean formula in CNF with n variables and m clauses such that each clause C_i has exactly four literals.
Output: A satisfying assignment, if exists, such that at least two literals in every clause C_i, for $1 \leq i \leq m$, are True.

Note: Write the reduction in the format described on the front page of this document.

Solution: To prove that Modified-4-SAT is NP-Complete, we need to prove it is both in NP and NP-Hard. Let's start with NP. The witness is a set of boolean assignments for the n variables. To prove that it is in NP, we simply need to loop over the m clauses. For each clause, we plug the corresponding assignments from the witness into the 4 literals. We then loop over the clause to check if 2 or more are true in which case we know the whole clause is true, but if only one or zero is true then the clause is false and the witness as a whole is false. This would be polytime because we are simply looping over the m clauses and then each clause which has a max length of 4, so it would run in $\mathcal{O}(m)$.
Now we need to prove it is NP-Hard by reducing a known NP-Complete problem to Modified-4-SAT in polytime. 1.) First we need to pick A, for this problem lets use 3SAT.
2.) To reduce 3SAT to Modified-4-SAT in polytime, we simply need to loop over all the m clauses in the input for 3SAT and add a literal to the end of the clause which we will call z which is always set to true. So an input clause of $(X_1 \vee X_2 \vee X_3)$ would turn into $(X_1 \vee X_2 \vee X_3 \vee Z)$ where Z is set to true.
3.) We now must prove that $x \in A \implies f(x) \in B$. For x to satisfy A which in our case is 3SAT, at least 1 of the 3 literals in each clause needs to be true. For our modified 4 sat, we need at least 2 of the literals in each clause to be true. So, if each clause has 3 literals and has 1, 2, or 3 trues, by adding another true to the clause, it would now have 4 literals and 2, 3, or 4 trues respectively which satisfies modified 4 sat. Therefore, if x satisfies 3sat, after our reduction it satisfies modified 4 sat.
4.) Lastly, we must prove Prove that $x \notin A \implies f(x) \notin B$. For x to not satisfy A which is 3SAT, it means that in at least one clause, none of the literals are true. Focusing on this false 3sat clause, if we apply our reduction to this clause, it would now have 3 falses and 1 true which does not satisfy our modified 4 sat which needs 2 trues in each clause. Therefore, if x is unsatisfiable for 3sat, adding a true to each clause, would make it unsatisfiable for modified 4 sat.
5.) Since SAT was NP-complete, and $SAT \leq_p modified4sat$, we can conclude that modified-4-SAT is NP-hard.

4 Problem 4 **30 / 30**

✓ **- 0 pts** *Correct*

- 30 pts Completely Incorrect/ Missing

Proof of NP

- 5 pts Incorrect/ Missing NP proof

- 3 pts Major Error

- 2 pts Minor Error

- 20 pts Reverse reduction provided (Modified-4-SAT $$\rightarrow$$ NP-complete problem)

Reduction

- 15 pts Incorrect Reduction

- 5 pts Minor Mistake (ex. small mistake in converting 3-SAT to Modified-4-SAT)

- 10 pts Major Error (ex. reduction cannot be justified)

Bi-directional Proof

- 5 pts Incorrect/ Missing $$x \in A \implies f(x) \in B$$

- 2 pts Minor Error (i.e. Incomplete explanation)

- 4 pts Major Error (i.e. forgot to prove it for all cases)

- 5 pts Incorrect/ Missing $$x \notin A \implies f(x) \notin B$$

- 2 pts Minor Error (i.e. Incomplete explanation)

- 4 pts Major Error (i.e. forgot to prove it for all cases)

gradescope

CS 3510: Design & Analysis of Algorithms 03/28/2023

Problem Set 7: NP-completeness I

Prof. Abrahim Ladha Due: 04/05/2023 11:59pm

- **Please TYPE your solutions using Latex or any other software. Handwritten solutions *won't* be accepted.**

Steps to write a reduction

To show that a problem is NP-complete, you need to show that the problem is both in NP and NP-hard. In a polynomial-time reduction, we have a *Problem B*, and we want to show that it is NP-complete. These are the steps:

- Demonstrate that *problem B* is in the class NP. This is a description of a procedure that verifies a candidate's solution. It needs to address the runtime. You should give a polytime verifier which takes as input a candidate problem, and a witness, and outputs true or false if the witness is a solution.
- Demonstrate that *problem B* is at least as hard as a problem previously proved to be NP-complete. Choose some NP-complete A and prove $A \leq_p B$. You need to show that *problem B* is NP-hard. This is done via polytime reduction from a known NP-complete *problem A* to the unknown *problem B* ($A \to B$) as follows:
  1. Choose A. You will have many NP-complete problems to pick from later on, and it is best to pick a similar one.
  2. Give your reduction f and argue that it runs in polynomial time.
  3. Prove that $x \in A \implies f(x) \in B$
  4. Prove that $x \notin A \implies f(x) \notin B$
  5. This is sufficient to show $x \in A \iff f(x) \in B$. Since A was NP-complete, and $A \leq_p B$, we can conclude that B is NP-hard. Note: You don't have to provide a formal proof. You can briefly explain in words both implications and why they hold.

The second bullet point above is prove that *problem B* is NP-hard which combined with the first bullet point yields the NP-complete proof.